

РОССИЙСКИЙ УНИВЕРСИТЕТ ДРУЖБЫ НАРОДОВ

Факультет физико-математических и естественных наук

Кафедра прикладной информатики и теории вероятностей

ОТЧЕТ

по лабораторной работе № 9

дисциплина: Архитектура компьютера

Студент: Венис Дмитрий Сергеевич

Группа: НКАбд-01-25

МОСКВА

2025 г.

Содержание

1. Цель работы
2. Теоретическое введение
3. Выполнение лабораторной работы
4. Выполнение самостоятельных заданий
5. Выводы

Список иллюстраций:

Рис.3.1.....	10
Рис.3.2.....	11
Рис.3.3.....	12
Рис.3.4.....	13
Рис.3.5.....	14
Рис.3.6.....	14
Рис.3.7.....	14
Рис.3.8.....	15
Рис.3.9.....	15
Рис.3.10.....	16
Рис.3.11.....	16
Рис.3.12.....	17
Рис.3.13.....	17
Рис.3.14.....	18
Рис.3.15.....	18
Рис.3.16.....	19
Рис.3.17.....	19
Рис.3.18.....	19
Рис.3.19.....	19
Рис.3.20.....	20
Рис.3.21.....	20
Рис.3.22.....	20
Рис.3.23.....	20
Рис.3.24.....	21
Рис.3.25.....	21
Рис.3.26.....	21
Рис.3.27.....	21
Рис.3.28.....	22
Рис.3.29.....	22
Рис.3.30.....	22
Рис.4.1.....	24
Рис.4.2.....	25

Рис.4.3.....	26
Рис.4.4.....	27
Рис.4.5.....	28
Рис.4.6.....	28

1.Цель работы

Приобретение навыков написания программ с использованием подпрограмм.

Знакомство с методами отладки при помощи GDB и его основными возможностями.

2. Теоретическое введение

Понятие об отладке

Отладка — это процесс поиска и исправления ошибок в программе. В общем случае его можно разделить на четыре этапа:

- обнаружение ошибки;
- поиск её местонахождения;
- определение причины ошибки;
- исправление ошибки.

Можно выделить следующие типы ошибок:

- синтаксические ошибки — обнаруживаются во время трансляции исходного кода и вызваны нарушением ожидаемой формы или структуры языка;
- семантические ошибки — являются логическими и приводят к тому, что программа запускается, отработывает, но не даёт желаемого результата;
- ошибки в процессе выполнения — не обнаруживаются при трансляции и вызывают прерывание выполнения программы (например, это ошибки, связанные с переполнением или делением на ноль).

Второй этап — поиск местонахождения ошибки. Некоторые ошибки обнаружить довольно трудно. Лучший способ найти место в программе, где находится ошибка, это разбить программу на части и произвести их отладку отдельно друг от друга. Третий этап — выяснение причины ошибки. После определения местонахождения ошибки обычно проще определить причину неправильной работы программы. Последний этап — исправление ошибки. После этого при повторном запуске программы, может обнаружиться следующая ошибка, и процесс отладки начнётся заново.

Методы отладки

Наиболее часто применяют следующие методы отладки:

- создание точек контроля значений на входе и выходе участка программы (например, вывод промежуточных значений на экран — так называемые диагностические сообщения);
- использование специальных программ-отладчиков.

Отладчики позволяют управлять ходом выполнения программы, контролировать и изменять данные. Это помогает быстрее найти место ошибки в программе и ускорить её исправление. Наиболее популярные способы работы с отладчиком — это использование точек останова и выполнение программы по шагам.

Пошаговое выполнение — это выполнение программы с остановкой после каждой строчки, чтобы программист мог проверить значения переменных и выполнить другие действия. Точки останова — это специально отмеченные места в программе, в которых программаотладчик приостанавливает выполнение программы и ждёт команд. Наиболее популярные виды точек останова:

- Breakpoint — точка останова (остановка происходит, когда выполнение доходит до определённой строки, адреса или процедуры, отмеченной программистом);
- Watchpoint — точка просмотра (выполнение программы приостанавливается, если программа обратилась к определённой переменной: либо считала её значение, либо изменила его).

Точки останова устанавливаются в отладчике на время сеанса работы с кодом программы, т.е. они сохраняются до выхода из программы-отладчика или до смены отлаживаемой программы.

Основные возможности отладчика GDB

GDB (GNU Debugger — отладчик проекта GNU) [1] работает на многих UNIX-подобных системах и умеет производить отладку многих языков программирования. GDB предлагает обширные средства для слежения и контроля за выполнением компьютерных программ. Отладчик не содержит собственного графического пользовательского интерфейса и использует стандартный текстовый интерфейс консоли. Однако для GDB существует несколько сторонних графических надстроек, а кроме того, некоторые интегрированные среды разработки используют его в качестве базовой подсистемы отладки. Отладчик GDB (как и любой другой отладчик) позволяет увидеть, что происходит «внутри» программы в момент её выполнения или что делает программа в момент сбоя. GDB может выполнять следующие действия:

- начать выполнение программы, задав всё, что может повлиять на её поведение;
- остановить программу при указанных условиях;
- исследовать, что случилось, когда программа остановилась;
- изменить программу так, чтобы можно было поэкспериментировать с устранением эффектов одной ошибки и продолжить выявление других.

Запуск отладчика GDB; выполнение программы; выход

Синтаксис команды для запуска отладчика имеет следующий вид:

```
gdb [опции] [имя_файла | ID процесса]
```

После запуска `gdb` выводит текстовое сообщение — так называемое «nice GDB logo». В следующей строке появляется приглашение (`gdb`) для ввода команд. Далее приведён список некоторых команд GDB. Команда `run` (сокращённо `r`) — запускает отлаживаемую программу в оболочке GDB. Если точки останова не были установлены, то программа выполняется и выводятся сообщения: `(gdb) run Starting program: test Program exited normally. (gdb)` Если точки останова были

заданы, то отладчик останавливается на соответствующей команде и выдаёт номер точки останова, адрес и дополнительную информацию — текущую строку, имя процедуры, и др. Команда `kill` (сокращённо `k`) прекращает отладку программы, после чего следует вопрос о прекращении процесса отладки: `Kill the program being debugged? (y or n)` у Если в ответ введено `y` (то есть «да»), отладка программы прекращается. Командой `run` её можно начать заново, при этом все точки останова (breakpoints), точки просмотра (watchpoints) и точки отлова (catchpoints) сохраняются. Для выхода из отладчика используется команда `quit` (или сокращённо `q`): `(gdb) q`

Если есть файл с исходным текстом программы, а в исполняемый файл включена информация о номерах строк исходного кода, то программу можно отлаживать, работая в отладчике непосредственно с её исходным текстом. Чтобы программу можно было отлаживать на уровне строк исходного кода, она должна быть откомпилирована с ключом `-g`. Посмотреть дизассемблированный код программы можно с помощью команды

`disassemble <метка/адрес>: (gdb) disassemble _start`

Существует два режима отображения синтаксиса машинных команд: режим Intel, используемый в том числе в NASM, и режим ATT (значительно отличающийся внешне). По умолчанию в дизассемблере GDB принят режим ATT. Переключиться на отображение команд с привычным Intel'овским синтаксисом можно, введя команду

`set disassembly-flavor intel`

Понятие подпрограммы

Подпрограмма — это, как правило, функционально законченный участок кода, который можно многократно вызывать из разных мест программы. В отличие от простых переходов из подпрограмм существует возврат на команду, следующую за вызовом. Если в программе встречается одинаковый участок кода, его можно оформить в виде подпрограммы, а во всех нужных местах поставить её вызов.

При этом подпрограмма будет содержаться в коде в одном экземпляре, что позволит уменьшить размер кода всей программы.

Инструкция call и инструкция ret

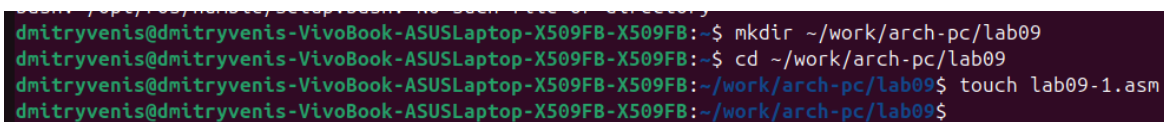
Для вызова подпрограммы из основной программы используется инструкция call, которая заносит адрес следующей инструкции в стек и загружает в регистр еір адрес соответствующей подпрограммы, осуществляя таким образом переход. Затем начинается выполнение подпрограммы, которая, в свою очередь, также может содержать подпрограммы. Подпрограмма завершается инструкцией ret, которая извлекает из стека адрес, занесённый туда соответствующей инструкцией call, и заносит его в еір. После этого выполнение основной программы возобновится с инструкции, следующей за инструкцией call.

Подпрограмма может вызываться как из внешнего файла, так и быть частью основной программы.

Важно помнить, что если в подпрограмме занести что-то в стек и не извлечь, то на вершине стека окажется не адрес возврата и это приведёт к ошибке выхода из подпрограммы. Кроме того, надо помнить, что подпрограмма без команды возврата не вернётся в точку вызова, а будет выполнять следующий за подпрограммой код, как будто он является её продолжением.

3.Выполнение лабораторной работы

1. Создадим каталог для выполнения лабораторной работы № 9, перейдем в него и создадим файл lab09-1.asm:



```
dmityvenis@dmityvenis-VivoBook-ASUSLaptop-X509FB-X509FB:~$ mkdir ~/work/arch-pc/lab09
dmityvenis@dmityvenis-VivoBook-ASUSLaptop-X509FB-X509FB:~$ cd ~/work/arch-pc/lab09
dmityvenis@dmityvenis-VivoBook-ASUSLaptop-X509FB-X509FB:~/work/arch-pc/lab09$ touch lab09-1.asm
dmityvenis@dmityvenis-VivoBook-ASUSLaptop-X509FB-X509FB:~/work/arch-pc/lab09$
```

Рис.3.1

2.Введем в файл lab09-1.asm текст программы из листинга 9.1. Создадим исполняемый файл и проверим его работу

```

/home/dmitryvenis/work/arch-pc/lab09/lab09-1.
#include 'in_out.asm'
SECTION .data
msg: DB 'Введите x: ',0
result: DB '2x+7=',0
SECTION .bss
x: RESB 80
res: RESB 80
SECTION .text
GLOBAL _start
_start:
;-----
; Основная программа
;-----
mov eax, msg
call sprint
mov ecx, x
mov edx, 80
call sread
mov eax,x
call atoi
call _calcul ; Вызов подпрограммы _calcul
mov eax,result
call sprint
mov eax,[res]
call iprintLF
call quit
;-----
; Подпрограмма вычисления
; выражения "2x+7"
_calcul:
mov ebx,2
mul ebx
add eax,7
mov [res],eax
ret ; выход из подпрограммы

```

Рис.3.2

```
dmityvenis@dmityvenis-VivoBook-ASUSLaptop-X509FB-X509FB:~/work/arch-pc/lab09$ nasm -f elf lab09-1.asm
dmityvenis@dmityvenis-VivoBook-ASUSLaptop-X509FB-X509FB:~/work/arch-pc/lab09$ ld -m elf_i386 -o lab09-1 lab09-1.o
dmityvenis@dmityvenis-VivoBook-ASUSLaptop-X509FB-X509FB:~/work/arch-pc/lab09$ ./lab09-1
Введите x: 10
2x+7=27
dmityvenis@dmityvenis-VivoBook-ASUSLaptop-X509FB-X509FB:~/work/arch-pc/lab09$
```

Рис.3.3

Изменим текст программы, добавив подпрограмму `_subcalcul` в подпрограмму `_calcul`, для вычисления выражения $f(g(x))$, где x вводится с клавиатуры, $f(x) = 2x + 7$, $g(x) = 3x - 1$.

```

/home/dmitryvenis/work/arch-pc/lab09/l
%include 'in_out.asm'

SECTION .data
    msg: DB 'Введите x: ',0
    result: DB 'f(g(x))=2(3x-1)+7=',0

SECTION .bss
    x: RESB 80
    res: RESB 80

SECTION .text
GLOBAL _start
_start:

    mov eax, msg
    call sprint

    mov ecx, x
    mov edx, 80
    call sread

    mov eax, x
    call atoi

    call _calcul

    mov eax, result
    call sprint
    mov eax, [res]
    call iprintLF

    call quit

_subcalcul:
    mov ebx, 3
    mul ebx
    sub eax, 1
    ret

_calcul:
    push eax
    call _subcalcul
    ....

```

Рис.3.4

```
_calcul:
    push eax
    call _subcalcul
    ....
    mov ebx,2
    mul ebx
    add eax,7
    ....
    mov [res],eax
    pop eax
    ret
```

1Help 2Save 3

Рис.3.5

```
dmityvenis@dmityvenis-VivoBook-ASUSLaptop-X509FB-X509FB:~/work/arch-pc/lab09$ nasm -f elf lab09-1.asm
dmityvenis@dmityvenis-VivoBook-ASUSLaptop-X509FB-X509FB:~/work/arch-pc/lab09$ ld -m elf_i386 -o lab09-1 lab09-1.o
dmityvenis@dmityvenis-VivoBook-ASUSLaptop-X509FB-X509FB:~/work/arch-pc/lab09$ ./lab09-1
Введите x: 10
f(g(x))=2(3x-1)+7=65
dmityvenis@dmityvenis-VivoBook-ASUSLaptop-X509FB-X509FB:~/work/arch-pc/lab09$
```

Рис.3.6

Создадим файл lab09-2.asm с текстом программы из Листинга 9.2. (Программа печати сообщения Hello world!):

```
dmityvenis@dmityvenis-VivoBook-ASUSLaptop-X509FB-X509FB:~/work/arch-pc/lab09$ touch lab09-2.asm
```

Рис.3.7

```

/home/dmitryvenis/work/a
SECTION .data
msg1: db "Hello, ",0x0
msg1Len: equ $ - msg1
msg2: db "world!",0xa
msg2Len: equ $ - msg2
SECTION .text
global _start
_start:
mov eax, 4
mov ebx, 1
mov ecx, msg1
mov edx, msg1Len
int 0x80
mov eax, 4
mov ebx, 1
mov ecx, msg2
mov edx, msg2Len
int 0x80
mov eax, 1
mov ebx, 0
int 0x80

```

Рис.3.8

Получим исполняемый файл. Для работы с GDB в исполняемый файл необходимо добавить отладочную информацию, для этого трансляцию программ необходимо проводить с ключом ‘-g’.

```

dmitryvenis@dmitryvenis-VivoBook-ASUSLaptop-X509FB-X509FB:~/work/arch-pc/lab09$ nasm -f elf -g -l lab09-2.lst lab09-2.asm
dmitryvenis@dmitryvenis-VivoBook-ASUSLaptop-X509FB-X509FB:~/work/arch-pc/lab09$ ld -m elf_i386 -o lab09-2 lab09-2.o
dmitryvenis@dmitryvenis-VivoBook-ASUSLaptop-X509FB-X509FB:~/work/arch-pc/lab09$

```

Рис.3.9

Загрузим исполняемый файл в отладчик gdb:


```

dmitryvenis@dmitryvenis-VivoBook-ASUSLaptop-X509FB-X509FB: ~/work/arch-pc/lab09$ gdb lab09-2
GNU gdb (Ubuntu 15.0.50.20240403-0ubuntu1) 15.0.50.20240403-git
Copyright (C) 2024 Free Software Foundation, Inc.
License GPLv3+: GNU GPL version 3 or later <http://gnu.org/licenses/gpl.html>
This is free software: you are free to change and redistribute it.
There is NO WARRANTY, to the extent permitted by law.
Type "show copying" and "show warranty" for details.
This GDB was configured as "x86_64-linux-gnu".
Type "show configuration" for configuration details.
For bug reporting instructions, please see:
<https://www.gnu.org/software/gdb/bugs/>.
Find the GDB manual and other documentation resources online at:
  <http://www.gnu.org/software/gdb/documentation/>.

For help, type "help".
Type "apropos word" to search for commands related to "word"...
Reading symbols from lab09-2...
(gdb) █

```

Рис.3.10

Проверим работу программы, запустив ее в оболочке GDB с помощью команды `run` (сокращённо `r`):

```

For help, type "help".
Type "apropos word" to search for commands related to "word"...
Reading symbols from lab09-2...
(gdb) run
Starting program: /home/dmitryvenis/work/arch-pc/lab09/lab09-2

This GDB supports auto-downloading debuginfo from the following URLs:
  <https://debuginfod.ubuntu.com>
Enable debuginfod for this session? (y or [n]) y
Debuginfod has been enabled.
To make this setting permanent, add 'set debuginfod enabled on' to .gdbinit.
Downloading separate debug info for system-supplied DSO at 0xf7ffc000
Hello, world!
[Inferior 1 (process 6510) exited normally]
(gdb) █

```

Рис.3.11

Для более подробного анализа программы установим брейкпоинт на метку `_start`, с которой начинается выполнение любой ассемблерной программы, и запустим её.

```
(gdb) break _start
Breakpoint 1 at 0x8049000: file lab09-2.asm, line 9.
(gdb) run
Starting program: /home/dmitryvenis/work/arch-pc/lab09/lab09-2

Breakpoint 1, _start () at lab09-2.asm:9
9      mov eax, 4
(gdb) █
```

Рис.3.12

Посмотрим дисассимилированный код программы с помощью команды `disassemble` начиная с метки `_start`

```
(gdb) disassemble _start
Dump of assembler code for function _start:
=> 0x08049000 <+0>:      mov     $0x4,%eax
0x08049005 <+5>:      mov     $0x1,%ebx
0x0804900a <+10>:     mov     $0x804a000,%ecx
0x0804900f <+15>:     mov     $0x8,%edx
0x08049014 <+20>:     int     $0x80
0x08049016 <+22>:     mov     $0x4,%eax
0x0804901b <+27>:     mov     $0x1,%ebx
0x08049020 <+32>:     mov     $0x804a008,%ecx
0x08049025 <+37>:     mov     $0x7,%edx
0x0804902a <+42>:     int     $0x80
0x0804902c <+44>:     mov     $0x1,%eax
0x08049031 <+49>:     mov     $0x0,%ebx
0x08049036 <+54>:     int     $0x80
End of assembler dump.
(gdb) █
```

Рис.3.13

Переключимся на отображение команд с Intel'овским синтаксисом, введя команду `set disassembly-flavor intel`

```

(gdb) set disassembly-flavor intel
(gdb) disassemble _start
Dump of assembler code for function _start:
=> 0x08049000 <+0>:    mov     eax,0x4
      0x08049005 <+5>:    mov     ebx,0x1
      0x0804900a <+10>:   mov     ecx,0x804a000
      0x0804900f <+15>:   mov     edx,0x8
      0x08049014 <+20>:   int     0x80
      0x08049016 <+22>:   mov     eax,0x4
      0x0804901b <+27>:   mov     ebx,0x1
      0x08049020 <+32>:   mov     ecx,0x804a008
      0x08049025 <+37>:   mov     edx,0x7
      0x0804902a <+42>:   int     0x80
      0x0804902c <+44>:   mov     eax,0x1
      0x08049031 <+49>:   mov     ebx,0x0
      0x08049036 <+54>:   int     0x80
End of assembler dump.
(gdb) █

```

Рис.3.14

Перечислите различия отображения синтаксиса машинных команд в режимах АТТ и Intel:

Ответ:

Основное различие между синтаксисом AT&T и Intel в GDB заключается в порядке операндов: в Intel используется порядок "назначение, источник" (например, `mov eax, 4`), тогда как в AT&T — "источник, назначение" (например, `mov $4, %eax`). Кроме того, в AT&T регистры обозначаются с префиксом `%`, а непосредственные значения с префиксом `$`, в то время как в Intel префиксы не используются. Обращение к памяти также отличается: в Intel используются квадратные скобки, а в AT&T — нет.

На предыдущих шагах была установлена точка останова по имени метки (`_start`). Проверим это с помощью команды `info breakpoints` (кратко `i b`):

```

(gdb) info breakpoints
Num   Type             Disp Enb Address      What
1     breakpoint       keep y   0x08049000 lab09-2.asm:9
      breakpoint already hit 1 time
(gdb)

```

Рис.3.15

Установим еще одну точку останова по адресу инструкции. Определим адрес предпоследней инструкции (`mov ebx,0x0`) и установим точку останова.

```
(gdb) break *0x8049031
Breakpoint 2 at 0x8049031: file lab09-2.asm, line 20.
(gdb)
```

Рис.3.16

Посмотрим информацию о всех установленных точках останова:

```
(gdb) i b
Num      Type      Disp Enb Address      What
1        breakpoint keep y 0x08049000 lab09-2.asm:9
          breakpoint already hit 1 time
2        breakpoint keep y 0x08049031 lab09-2.asm:20
(gdb)
```

Рис.3.17

Отладчик может показывать содержимое ячеек памяти и регистров, а при необходимости позволяет вручную изменять значения регистров и переменных. Выполним 5 инструкций с помощью команды `stepi` (или `si`) и проследим за изменением значений регистров. Значения каких регистров изменяются?

```
(gdb) si
(gdb) si
(gdb) si
(gdb) si
(gdb) si
(gdb) █
```

Рис.3.18

Ответ: Изменяются значения регистров: `eax`, `ebx`, `ecx`, `edx`

С помощью команды `x <имя переменной>` также можно посмотреть содержимое переменной. Посмотрим значение переменной `msg1` по имени

```
(gdb) x/1sb &msg1
0x804a000 <msg1>: "Hello, "
(gdb)
```

Рис.3.19

Посмотрим значение переменной msg2 по адресу

```
(gdb) x/1sb 0x804a008
0x804a008 <msg2>: "world!\n\034"
(gdb)
```

Рис.3.20

Изменим первый символ переменной msg1

```
(gdb) set {char}&msg1='h'
(gdb) set {char}0x804a001='h'
(gdb) x/1sb &msg1
0x804a000 <msg1>: "hhllo, "
(gdb)
```

Рис.3.21

Заменяем любой символ во второй переменной msg2

```
(gdb) set {char}&msg2='0'
(gdb) set {char}0x804a001='0'
(gdb) x/1sb &msg2
0x804a008 <msg2>: "0orld!\n\034"
(gdb)
```

Рис.3.22

Выведем в различных форматах (в шестнадцатеричном формате, в двоичном формате и в символьном виде) значение регистра edx.

```
(gdb) p/x $edx
$4 = 0x8
(gdb) p/t $edx
$5 = 1000
(gdb) p/c $edx
$6 = 8 '\b'
(gdb)
```

Рис.3.23

С помощью команды set изменим значение регистра ebx:

```
(gdb) set $ebx='2'  
(gdb) p/s $ebx  
$1 = 50
```

Рис.3.24

```
(gdb) set $ebx=2  
(gdb) p/s $ebx  
$3 = 2  
(gdb) █
```

Рис.3.25

Объясните разницу вывода команд p/s \$ebx.

Ответ: В первом случае в регистр ebx записывается ASCII код символа 2, а во втором случае в регистр ebx записывается числовое значение 2

Завершим выполнение программы с помощью команды continue (сокращенно c) или stepi (сокращенно si) и выйдем из GDB с помощью команды quit (сокращенно q).

```
dmityvenis@dmityvenis-VivoBook-ASUSLaptop-X509FB-X509FB:~/work/arch-pc/lab09$ █
```

Рис.3.26

Скопируем файл lab8-2.asm, созданный при выполнении лабораторной работы №8, с программой выводящей на экран аргументы командной строки (Листинг 8.2) в файл с именем lab09-3.asm:

```
dmityvenis@dmityvenis-VivoBook-ASUSLaptop-X509FB-X509FB:~/work/arch-pc/lab09$ cp ~/work/arch-pc/lab08/lab8-2.asm ~/work/arch-pc/lab09/lab09-3.asm
```

Рис.3.27

Создадим исполняемый файл.

```

dmitryvenis@dmitryvenis-VivoBook-ASUSLaptop-X509FB-X509FB: ~/work/arch-pc/lab09$ nasm -f elf -g -l lab09-3.lst lab09-3.asm
dmitryvenis@dmitryvenis-VivoBook-ASUSLaptop-X509FB-X509FB: ~/work/arch-pc/lab09$ ld -m elf_i386 -o lab09-3 lab09-3.o
dmitryvenis@dmitryvenis-VivoBook-ASUSLaptop-X509FB-X509FB: ~/work/arch-pc/lab09$ gdb --args lab09-3 аргумент1 аргумент 2 'аргумент 3'
GNU gdb (Ubuntu 15.0.50.20240403-0ubuntu1) 15.0.50.20240403-git
Copyright (C) 2024 Free Software Foundation, Inc.
License GPLv3+: GNU GPL version 3 or later <http://gnu.org/licenses/gpl.html>
This is free software: you are free to change and redistribute it.
There is NO WARRANTY, to the extent permitted by law.
Type "show copying" and "show warranty" for details.
This GDB was configured as "x86_64-linux-gnu".
Type "show configuration" for configuration details.
For bug reporting instructions, please see:
<https://www.gnu.org/software/gdb/bugs/>.
Find the GDB manual and other documentation resources online at:
<http://www.gnu.org/software/gdb/documentation/>.

For help, type "help".
Type "apropos word" to search for commands related to "word"...
Reading symbols from lab09-3...
(gdb)

```

Рис.3.28

Загрузим исполняемый файл в отладчик, указав аргументы:

```

(gdb) b _start
Breakpoint 1 at 0x80490e8: file lab09-3.asm, line 8.
(gdb) run
Starting program: /home/dmitryvenis/work/arch-pc/lab09/lab09-3 аргумент1 аргумент 2 аргумент\ 3

This GDB supports auto-downloading debuginfo from the following URLs:
  <https://debuginfod.ubuntu.com>
Enable debuginfod for this session? (y or [n]) y
Debuginfod has been enabled.
To make this setting permanent, add 'set debuginfod enabled on' to .gdbinit.
Downloading separate debug info for system-supplied DSO at 0xf7ffc000

Breakpoint 1, _start () at lab09-3.asm:8
8      pop ecx ; Извлекаем из стека в 'ecx' количество
(gdb)

```

Рис.3.29

Посмотрим остальные позиции стека – по адресу [esp+4] располагается адрес в памяти где находится имя программы, по адресу [esp+8] храниться адрес первого аргумента, по адресу [esp+12] – второго и т.д

```

(gdb) x/x $esp
0xffffcf50: 0x00000005
(gdb) x/s *(void**)(esp + 4)
0xffffd12b: "/home/dmitryvenis/work/arch-pc/lab09/lab09-3"
(gdb) x/s *(void**)(esp + 8)
0xffffd158: "аргумент1"
(gdb) x/s *(void**)(esp + 12)
0xffffd16a: "аргумент"
(gdb) x/s *(void**)(esp + 16)
0xffffd17b: "2"
(gdb) x/s *(void**)(esp + 20)
0xffffd17d: "аргумент 3"
(gdb) x/s *(void**)(esp + 24)
0x0: <error: Cannot access memory at address 0x0>
(gdb)

```

Рис.3.30

Объясните, почему шаг изменения адреса равен 4 ([esp+4], [esp+8], [esp+12] и т.д.).

Ответ: Так как в архитектуре x86 стандартный размер адреса составляет 32 бита, то его размер составляет 4 байта. Поскольку каждый элемент занимает ровно 4 байта, чтобы перейти к следующему элементу, необходимо увеличить смещение на 4.

Вывод: Выполнили задания лабораторной работы

4.Выполнение самостоятельной работы

1. Преобразуйте программу из лабораторной работы №8 (Задание №1 для самостоятельной работы), реализовав вычисление значения функции $f(x)$ как подпрограмму

Ответ:

```
/home/dmitryvenis/work/arch-pc/lab09/main-1.asm [-M--] 7 L:[ 1+37 38/ 38] *(538 / 538b) <EOF>
%include 'in_out.asm'

SECTION .data
msg: DB 'Введите x: ', 0
res_msg: DB 'f(x) = (x+1)*7. Результат: ', 0

SECTION .bss
x: RESB 80
res: RESB 80

SECTION .text
GLOBAL _start

_start:
    mov eax, msg
    call sprint
    mov ecx, x
    mov edx, 80
    call sread
    mov eax, x
    call atoi.....
    ....
    call _calc_func.
    ....
    mov [res], eax..
    ....
    mov eax, res_msg
    call sprint
    mov eax, [res]
    call iprintLF
    call quit

_calc_func:
    add eax, 1...
    mov ebx, 7...
    mul ebx.....
    ....
    ret
```

1Help 2Save 3Mark 4Replac 5Copy 6Move

Рис.4.1

```
dmityvenis@dmityvenis-VivoBook-ASUSLaptop-X509FB-X509FB: ~/work/arch-pc/lab09$ nasm -f elf main-1.asm
dmityvenis@dmityvenis-VivoBook-ASUSLaptop-X509FB-X509FB: ~/work/arch-pc/lab09$ ld -m elf_i386 -o main-1 main-1.o
dmityvenis@dmityvenis-VivoBook-ASUSLaptop-X509FB-X509FB: ~/work/arch-pc/lab09$ ./main-1
Введите x: 10
f(x) = (x+1)*7. Результат: 77
dmityvenis@dmityvenis-VivoBook-ASUSLaptop-X509FB-X509FB: ~/work/arch-pc/lab09$
```

Рис.4.2

2. В листинге 9.3 приведена программа вычисления выражения $(3 + 2) * 4 + 5$.

При запуске данная программа дает неверный результат. Проверьте это. С помощью отладчика GDB, анализируя изменения значений регистров, определите ошибку и исправьте ее

Ответ:

Создадим файл lab09-4.asm и проверим его на наличие ошибок с помощью GDB

```
/home/dmitryvenis/work/arch-p
#include 'in_out.asm'

SECTION .data
div: DB 'Результат: ',0
SECTION .text
GLOBAL _start
_start:
mov ebx,3
mov eax,2
add ebx,eax
mov ecx,4
mul ecx
add ebx,5
mov edi,ebx
mov eax,div
call sprint
mov eax,edi
call iprintLF

call quit
```

Puc.4.3

```
dmitryvenis@dmitryvenis-VivoBook-ASUSLaptop-X509FB-X509FB: ~/work/arch-pc/lab09
0x80490f2 <_start+10> add    %eax,%ebx
B+ 0x80490e8 <_start>    mov    $0x3,%ebx
0x80490ed <_start+5>    mov    $0x2,%eax
0x80490f2 <_start+10> add    %eax,%ebx
0x80490f4 <_start+12> mov    $0x4,%ecx
0x80490f9 <_start+17> mul    %ecx
->0x80490fb <_start+19> add    $0x5,%ebx
0x80490fe <_start+22> mov    %ebx,%edi
0x8049100 <_start+24> mov    $0x804a000,%eax
0x8049105 <_start+29> call   0x804900f <sprint>
0x804910a <_start+34> mov    %edi,%eax
0x804910c <_start+36> call   0x8049086 <iprintf>
0x8049111 <_start+41> call   0x80490db <quit>
0x8049116      add    %al,(%eax)
0x8049118      add    %al,(%eax)
0x804911a      add    %al,(%eax)
0x804911c      add    %al,(%eax)
0x804911e      add    %al,(%eax)
0x8049120      add    %al,(%eax)
0x8049122      add    %al,(%eax)
0x8049124      add    %al,(%eax)
0x8049126      add    %al,(%eax)
0x8049128      add    %al,(%eax)
0x804912a      add    %al,(%eax)
0x804912c      add    %al,(%eax)
0x804912e      add    %al,(%eax)
0x8049130      add    %al,(%eax)
0x8049132      add    %al,(%eax)
(gdb) run
(gdb) process 20220 (asm) In: _start
(gdb) info breakpoints
Num   Type             Disp Enb Address      What
1     breakpoint       keep y   <PENDING>   _mistake
(gdb) break _start
Breakpoint 2 at 0x80490e8: file lab09-4.asm, line 8.
(gdb) run
Starting program: /home/dmitryvenis/work/arch-pc/lab09/lab09-4

Breakpoint 2, _start () at lab09-4.asm:8
(gdb) si
(gdb) si
(gdb) si
(gdb) si
(gdb) si
(gdb) 
```

Рис.4.4

Выполним программу по шагам и проследим за регистрами.

Ошибка: инструкция mul ecx умножает значение в eax (которое равно 2) на ecx (4), получается 8. Но нам нужно умножить результат сложения (5), который хранится в ebx.

Результат: Программа выдает 10 вместо 25.

Исправим программу

```

/home/dmitryvenis/work/arch-pc/lab09/lab09-4.asm [-M--] 13 L:[ 1+29 30/ 30] *(786 / 786b) <EOF>
#include 'in_out.asm'

SECTION .data
div: DB 'Результат: ', 0

SECTION .text
GLOBAL _start ; Обязательное объявление глобальной точки входа

_start:        ; <-- Метка начала выполнения программы
; Вычисление выражения (3+2)*4+5 = 25
mov ebx, 3
mov eax, 2
add ebx, eax    ; EBX = 5
....
mov eax, ebx    ; ИСПРАВЛЕНИЕ: Перенос 5 в EAX для MUL
....
mov ecx, 4
mul ecx         ; EAX = 5 * 4 = 20
....
add eax, 5      ; EAX = 20 + 5 = 25
....
mov edi, eax    ; Сохраняем результат в EDI

; Вывод результата
mov eax, div
call sprint
mov eax, edi
call iprintLF
....
call quit

```

Рис.4.5

```

dmitryvenis@dmitryvenis-VivoBook-ASUSLaptop-X509FB-X509FB:~/work/arch-pc/lab09$ nasm -f elf lab09-4.asm
dmitryvenis@dmitryvenis-VivoBook-ASUSLaptop-X509FB-X509FB:~/work/arch-pc/lab09$ ld -m elf_i386 -o lab09-4 lab09-4.o
dmitryvenis@dmitryvenis-VivoBook-ASUSLaptop-X509FB-X509FB:~/work/arch-pc/lab09$ ./lab09-4
Результат: 25
dmitryvenis@dmitryvenis-VivoBook-ASUSLaptop-X509FB-X509FB:~/work/arch-pc/lab09$

```

Рис.4.6

Все работает исправно

5.Выводы

Я приобрел навыки написания программ с использованием подпрограмм.
Познакомился с методами отладки при помощи GDB и его основными возможностями.